UNITÀ 3 · LA COMUNICAZIONE WEBSOCKET TRA BROWSER E SERVER

Il sondaggio in diretta (anche in gruppo)

Una domanda, tre risposte e barre che crescono in tempo reale su tutti gli schermi collegati ogni volta che qualcuno vota.

Cosa consegni: un file **sondaggio-cognome.zip** con dentro **SondaggioEndpoint.java** (oppure il server alternativo modificato), **index.html** e **prova.pdf** (screenshot di due browser e della scheda Rete di DevTools, con una breve spiegazione). **Lavoro di gruppo facoltativo**, al massimo 3 persone: consegna una sola persona con i nomi di tutti in prova.pdf. **Scadenza:** quella indicata su Novaula.

Che cosa devi fare

1. **Il protocollo.** Il browser manda `VOTO A`, `VOTO B` o `VOTO C`. Il server, dopo ogni voto e quando si apre una connessione, manda a tutti il conteggio in JSON: `{"A":3,"B":1,"C":0}`.
2. **Il server su Tomcat.** Parti da `ChatEndpoint` del laboratorio: `@ServerEndpoint("/ws")`, un `Set<Session>` e una `ConcurrentHashMap<String, AtomicInteger>` con i tre contatori. Ignora i messaggi che non sono un voto valido.
3. **La pagina.** Tre pulsanti e tre barre (`div` con larghezza in percentuale). In `onmessage` usa `JSON.parse(e.data)` e aggiorna le barre e i numeri.
4. **Un voto a testa.** Dopo il voto la pagina disattiva i pulsanti. Nella prova spiega perché questo controllo, fatto solo nel browser, non basta e che cosa dovrebbe fare il server.
5. **La prova.** Due finestre affiancate che mostrano gli stessi numeri, e la scheda Rete con l'handshake (stato 101) e i frame inviati e ricevuti.

Un aiuto per il server

```
@OnMessage
public void messaggio(String testo, Session da) {
    if (!testo.matches("VOTO [ABC]")) return;            // input non valido: ignorato
    voti.get(testo.substring(5)).incrementAndGet();
    inviaATutti(conteggi());   // conteggi() costruisce il JSON con String.format
}
```

Senza Tomcat?

Se a casa non riesci a installare Tomcat, usa il server alternativo **MiniWebSocket.java** nella pagina seguente: usa solo il JDK, si avvia con `java MiniWebSocket.java` accanto a `index.html` e risponde su `http://localhost:8081/`. In `gestisci`, al posto di `broadcast(...)`, conta il voto e manda i conteggi.

Come viene valutato

Criterio	Punti
Server corretto: contatori thread-safe, validazione dei messaggi, invio a tutti, conteggi alla connessione	4
Pagina: WebSocket, JSON.parse, barre aggiornate in tempo reale, textContent per i testi	3
Prova: due browser sincronizzati e handshake 101 mostrato e spiegato	2
Riflessione sul "un voto a testa" (perché il controllo va fatto sul server)	1

Appendice · MiniWebSocket.java (solo JDK, per il laboratorio)

Gestisce l'handshake, i frame di testo fino a 64 KB e la chiusura. Non usa TLS e non controlla l'header Origin: va bene per imparare, non per un servizio vero.

```
import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import static java.nio.charset.StandardCharsets.UTF_8;
/** Server WebSocket minimo, solo JDK: handshake, frame di testo brevi (< 64 KB), broadcast. Solo per il laboratorio. */
public class MiniWebSocket {
    static final Set<OutputStream> clienti = ConcurrentHashMap.newKeySet();
    public static void main(String[] args) throws IOException {
        int porta = args.length > 0 ? Integer.parseInt(args[0]) : 8081;
        try (ServerSocket server = new ServerSocket(porta)) {
            System.out.println("Apri http://localhost:" + porta + "/");
            while (true) {
                Socket s = server.accept();
                Thread.ofVirtual().start(() -> gestisci(s));
            }
        }
    }
    static void gestisci(Socket s) {
        OutputStream out = null;
        try (s) {
            InputStream in = s.getInputStream();
            out = s.getOutputStream();
            String key = null, riga;
            while (!(riga = leggiRiga(in)).isEmpty())                       // header HTTP
                if (riga.toLowerCase().startsWith("sec-websocket-key:")) key = riga.substring(18).trim();
            if (key == null) {                                              // richiesta HTTP normale: manda la pagina
                byte[] html = new FileInputStream("index.html").readAllBytes();
                out.write(("HTTP/1.1 200 OK\r\nContent-Type: text/html; charset=utf-8\r\nContent-Length: "
                        + html.length + "\r\nConnection: close\r\n\r\n").getBytes(UTF_8));
                out.write(html);
                return;
            }
            out.write(("HTTP/1.1 101 Switching Protocols\r\nUpgrade: websocket\r\nConnection: Upgrade\r\n"
                    + "Sec-WebSocket-Accept: " + accept(key) + "\r\n\r\n").getBytes(UTF_8));
            clienti.add(out);
            while (true) {
                int b0 = in.read(), b1 = in.read();
                if (b0 < 0 || (b0 & 0x0F) == 8) break;                      // fine stream o frame close
                int len = b1 & 0x7F;
                if (len == 126) len = (in.read() << 8) | in.read();
                byte[] mask = in.readNBytes(4), dati = in.readNBytes(len);
                for (int i = 0; i < len; i++) dati[i] ^= mask[i % 4];       // il client maschera sempre
                if ((b0 & 0x0F) == 1) broadcast(new String(dati, UTF_8));
            }
        } catch (Exception e) {
            System.out.println("Connessione chiusa: " + e.getMessage());
        } finally {
            if (out != null) clienti.remove(out);
        }
    }
    static void broadcast(String testo) {
        byte[] d = testo.getBytes(UTF_8);
        ByteArrayOutputStream f = new ByteArrayOutputStream();
        f.write(0x81);                                                      // FIN + opcode 1 (testo)
        if (d.length < 126) f.write(d.length);
        else { f.write(126); f.write(d.length >> 8); f.write(d.length & 0xFF); }
        f.writeBytes(d);                                                    // dal server: niente maschera
        for (OutputStream o : clienti) {
            try { synchronized (o) { o.write(f.toByteArray()); o.flush(); } }
            catch (IOException e) { clienti.remove(o); }
        }
    }
    static String accept(String key) throws Exception {                     // RFC 6455
        byte[] sha1 = java.security.MessageDigest.getInstance("SHA-1")
                .digest((key + "258EAFA5-E914-47DA-95CA-C5AB0DC85B11").getBytes(UTF_8));
        return Base64.getEncoder().encodeToString(sha1);
    }
    static String leggiRiga(InputStream in) throws IOException {
        StringBuilder sb = new StringBuilder();
        int c;
        while ((c = in.read()) != -1 && c != '\n') if (c != '\r') sb.append((char) c);
        return sb.toString();
    }
}
```